CM12002
Computer Systems Architectures

Control Instructions

Fabio Nemetz

Summary

In this lecture:

1. Design philosophies for instruction sets: RISC vs. CISC

2. Control instructions:
   a) unconditional branching
   b) conditional branching and
   c) subroutines

1. Design philosophies for instruction sets: CISC vs RISC

CISC: Complex Instruction Set Computer

- There is a trend to richer instruction sets which include a larger and more complex number of instructions

- Two principal reasons for this trend:
  - A desire to simplify compilers
  - A desire to improve performance

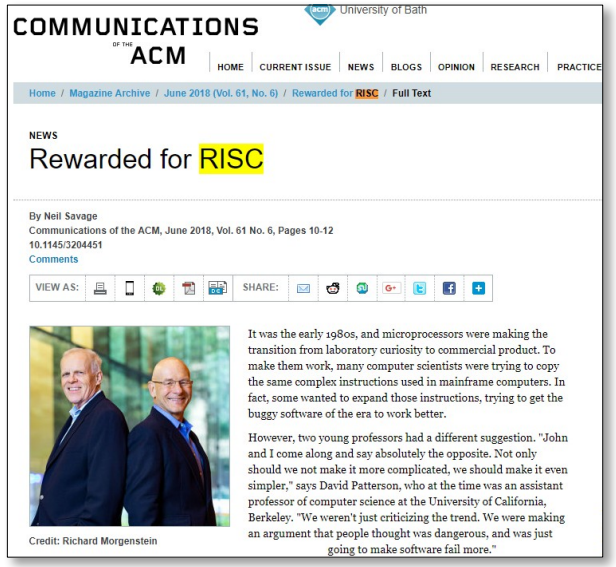
- There are two advantages to smaller programs:
  - The program takes up less memory
  - Should improve performance
    - Fewer instructions means fewer instruction bytes to be fetched
    - More instructions fit in cache(s)

1. Design philosophies for instruction sets: CISC vs RISC

CISC: Complex Instruction Set Computer

RISC: Reduced Instruction Set Computer

CISC	RISC
More powerful instructions	Simpler instructions
More complicated to decode instructions	Simple decoding
Arithmetic instructions also perform memory accesses (register memory architecture)	Arithmetic instructions only use registers (load/store architecture)
Less registers	More registers


COMMUNICATIONS OF THE ACM

University of Bath

HOME | CURRENT ISSUE | NEWS | BLOGS | OPINION | RESEARCH | PRACTICE

Home / Magazine Archive / June 2018 (Vol. 61, No. 6) / Rewarded for RISC / Full Text

NEWS

Rewarded for RISC

By Neil Savage
Communications of the ACM, June 2018, Vol. 61 No. 6, Pages 10-12
10.1145/3204451
Comments

VIEW AS: SHARE:

It was the early 1980s, and microprocessors were making the transition from laboratory curiosity to commercial product. To make them work, many computer scientists were trying to copy the same complex instructions used in mainframe computers. In fact, some wanted to expand those instructions, trying to get the buggy software of the era to work better.

However, two young professors had a different suggestion. "John and I come along and say absolutely the opposite. Not only should we not make it more complicated, we should make it even simpler," says David Patterson, who at the time was an assistant professor of computer science at the University of California, Berkeley. "We weren't just criticizing the trend. We were making an argument that people thought was dangerous, and was just going to make software fail more."

Credit: Richard Morgenstein

https://goo.gl/Bmor6i

David Patterson and John Hennessy

CISC: Complex Instruction Set Computer

RISC: Reduced Instruction Set Computer

- RISC is relatively straightforward due to having fewer and simpler instructions
- Each instruction only requires one clock cycle to execute, so instructions are executed uniformly*.
- However, as there are more lines of code, more memory is needed to store the assembly level instructions
- Compiler needs to perform more work to convert high-level language code into RISC instruction statements

- CISC instructions complete a task in fewer lines of assembly code than RISC
- As CISC code length is quite short, not a lot of memory is needed to store instructions
- CISC: the number of general-purpose registers that can be fitted into the processor is lower because decoding instructions require more transistors

Cs.stanford.edu. 2000. RISC Vs. CISC. Available at: https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/risccisc/

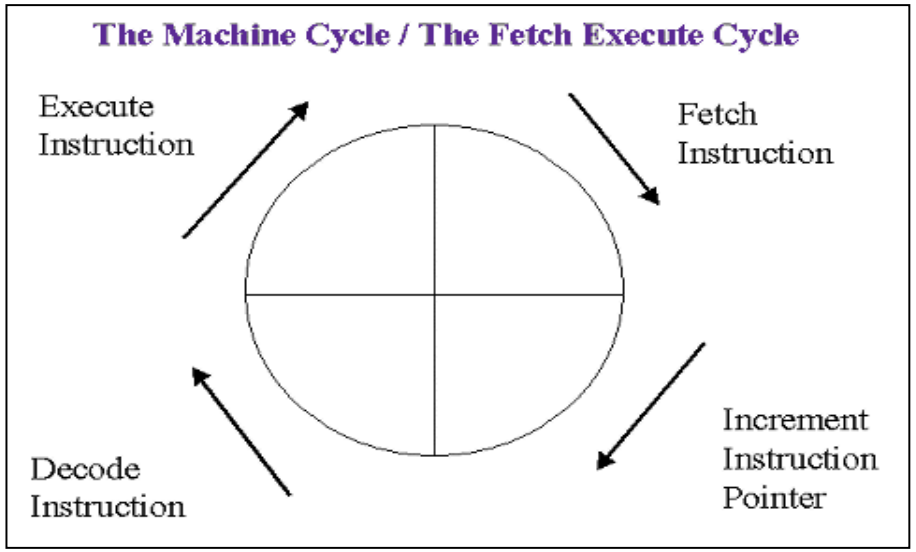
*As a result, pipelining is easy (pipelining will be covered in future lectures)

CISC (x86, MC68000, 6502, AMD…)

RISC (ARM, MIPS, PowerPC, …)

2. Control instructions

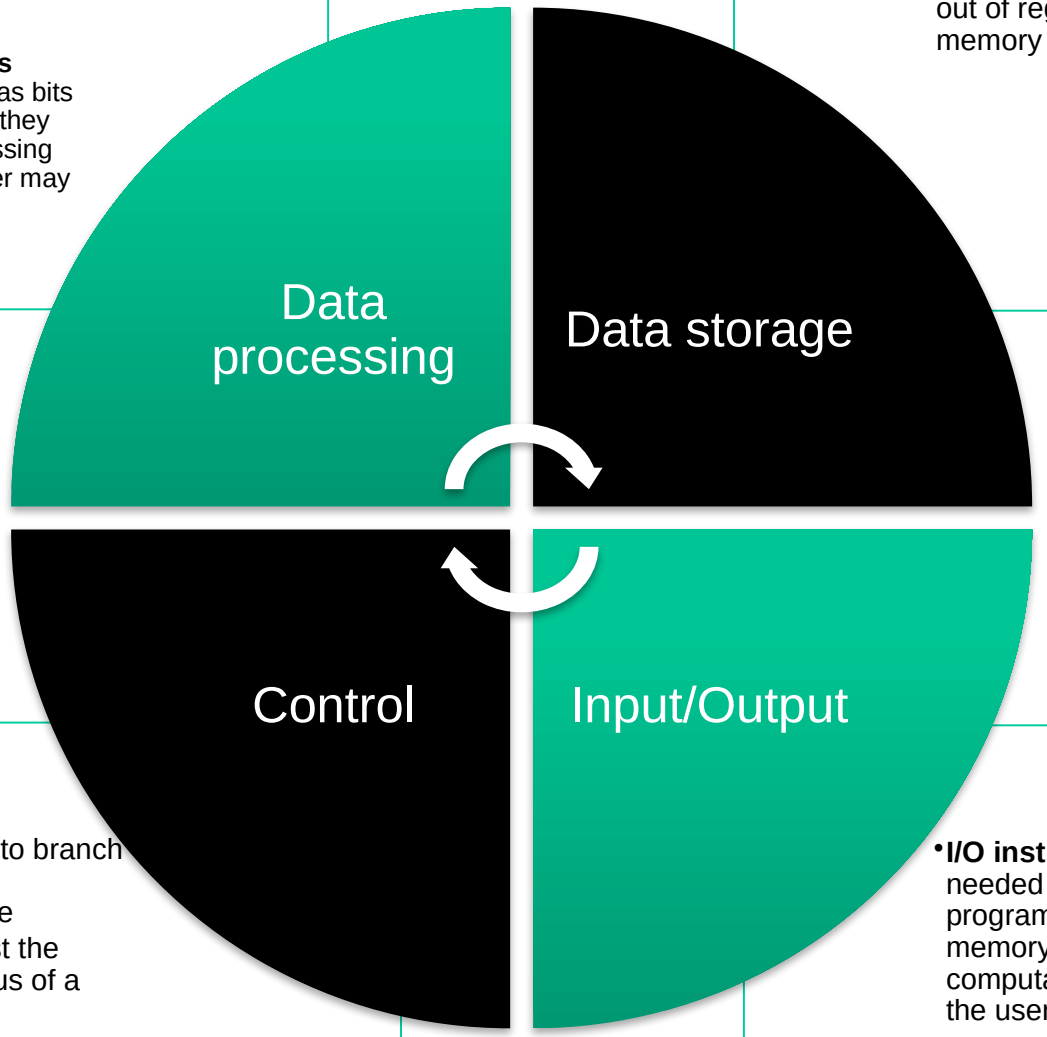
- Normally instructions are executed sequentially



Instruction Types

- **Arithmetic instructions** provide computational capabilities for processing numeric data
- **Logic (Boolean) instructions** operate on the bits of a word as bits rather than as numbers, thus they provide capabilities for processing any other type of data the user may wish to employ

- **Movement of data** into or out of register and or memory locations



- **Control instructions** are used to branch to a different set of instructions depending on the decision made
- Test instructions are used to test the value of a data word or the status of a computation

- **I/O instructions** are needed to transfer programs and data into memory and the results of computations back out to the user

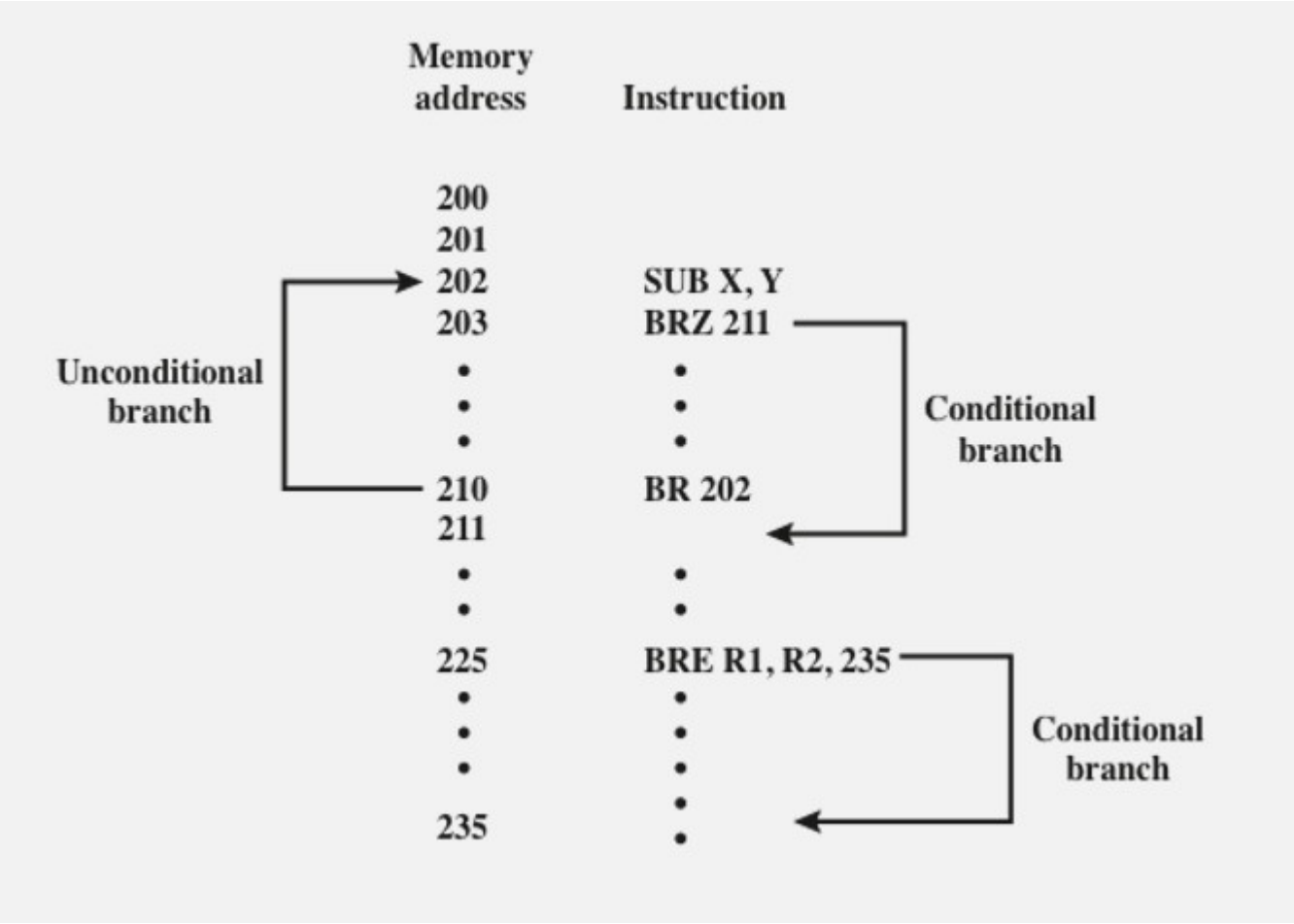
Control or transfer-of-control instructions

- These instructions can alter the contents of the PC (program counter): jump (or branch) and subroutine

- Reasons why control instructions are required:
  - It is essential to be able to execute each instruction more than once
  - Virtually all programs involve some decision making
  - It helps if there are mechanisms for breaking the task up into smaller pieces that can be worked on one at a time

- Most common control instructions found in instruction sets:
  - Jump (Branch)
    - Unconditional
    - Conditional
  - Subroutines (Procedure call)

Conditional and Unconditional Branch



A branch/jump instruction has as one of its operands the address of the next instruction to be executed.

Most often, the instruction is a conditional branch instruction. That is, the branch is made (update program counter to equal address specified in operand) only if a certain condition is met. Otherwise, the next instruction in sequence is executed (increment program counter as usual).

A branch instruction in which the branch is always taken is an unconditional branch.

Figure shows examples of these operations. Note that a branch can be either forward (an instruction with a higher address) or backward (lower address).

The example shows how an unconditional and a conditional branch can be used to create a repeating loop of instructions. The instructions in locations 202 through 210 will be executed repeatedly until the result of subtracting Y from X is 0.

Source: Stallings book (fig 13.7)

Let’s think high level (in C)

Given the C code below, we need to think how an if-then-else can be implemented in low-level

```
if (i==j)
    h = i + j;
else
    h = i-j;

...
```

```
Load registers (R1 = i; R2 = j; R3 = h;)
CMP R1,R2
JNE DoElse
    ADD R1,R2
    MOVE R3,R1
    JMP SkipElse
DoElse:
    SUB R1,R2
    MOVE R3,R1

SkipElse:
...
```

JNE = jump if not equal (conditional branch)

JMP = unconditional branch

2. Control instructions

Unconditional branch (jump)

- The execute phase of these operations takes the operand address and transfers it to the PC.

- The next operation then comes from the location addressed by the PC, in the normal way.

Example (x86)

```
     MOVE ECX, DWORD
      ; some code here
L1:  DEC ECX        ; ECX = ECX -1
     ….
     ….
     JMP L1         ; Loads PC with the address of L1
```

JMP: jump

2. Control instructions

Conditional branch

- we can arrange to branch (by changing the PC contents) only if the condition is TRUE. Then we need only *one* explicit operand.

2. Control instructions

condition or result	x86	PDP-11, VAX	ARM
zero (implies equal for sub/cmp)	JZ; JNZ; JE; JNE	BEQ; BNE	BEQ; BNE

Example (x86)

```
        MOVE ECX, DWORD
L1:      ; some code here
        DEC ECX          ; ECX = ECX -1
        CMP ECX, 33h     ; compare ECX with 33h
        JNE L1           ; Loads PC with address of L1, if operands of
                           previous CMP instruction are not equal.
        …
```

JNE: jump if not equal

2. Control instructions

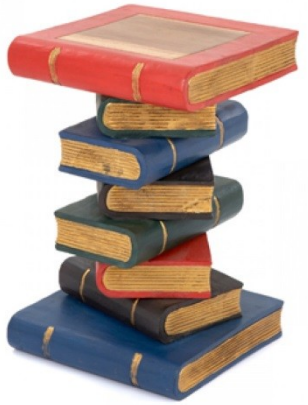
- Conditional branches can come in a variety of forms. One-bit *flags* or *condition codes* are used to denote the state of various aspects of the ALU:

CF - carry flag:	Set on bit carry; cleared otherwise
PF - parity flag:	Set if low-order eight bits of result contain an even number of "1" bits; cleared otherwise
ZF - zero flags:	Set if result is zero; cleared otherwise
SF - sign flag:	Set equal to high-order bit of result (0 if positive, 1 if negative)
OF - overflow flag:	Set if result is too large a positive number or too small a negative number (excluding sign bit) to fit in destination operand; cleared otherwise

- Conditional branches allow the programmer to construct the programming control structures that make computers so powerful:
  - *Decisions*: if a condition is true do X, else do Y;
  - *Loops*: do Z a fixed number of times; or do Z until a condition is true; or do Z while a condition is true.

2. Control instructions: **Subroutines**

- Repeated tasks may be implemented in a more structured way as *subroutines* or *procedures (aka* **procedure**, a **function**, a **routine**, a **method**, or a **subprogram)**
  - **function** → in C
  - **method** → in java
- A block of instructions for performing the task is placed in the memory and the subroutine is *called* by branching to the starting location of the block.
- **When the block of instructions is completed, the subroutine *returns* to the point at which it was called.**
- This can happen many times within a program.
- To accomplish that, we need the concept of a **STACK**

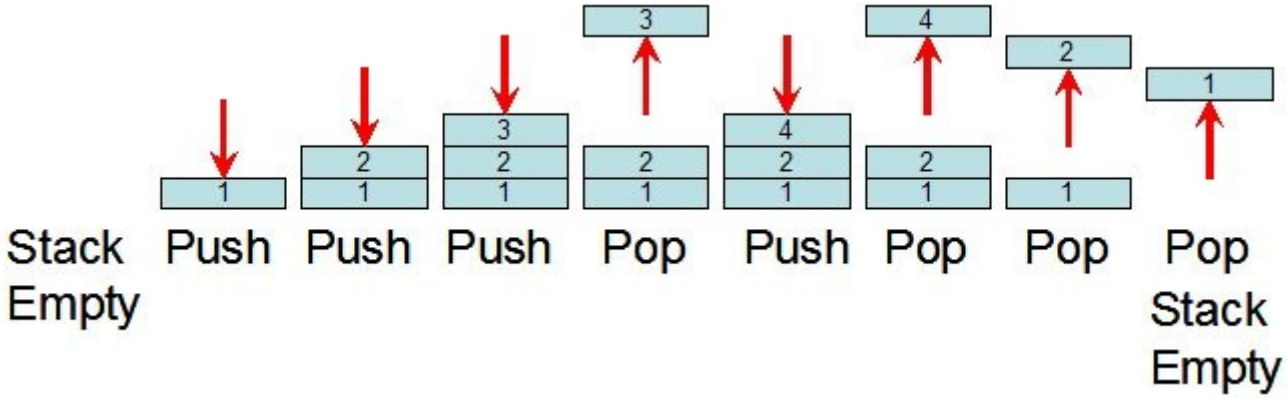


Stack

“Last in First out” (LIFO) principle.

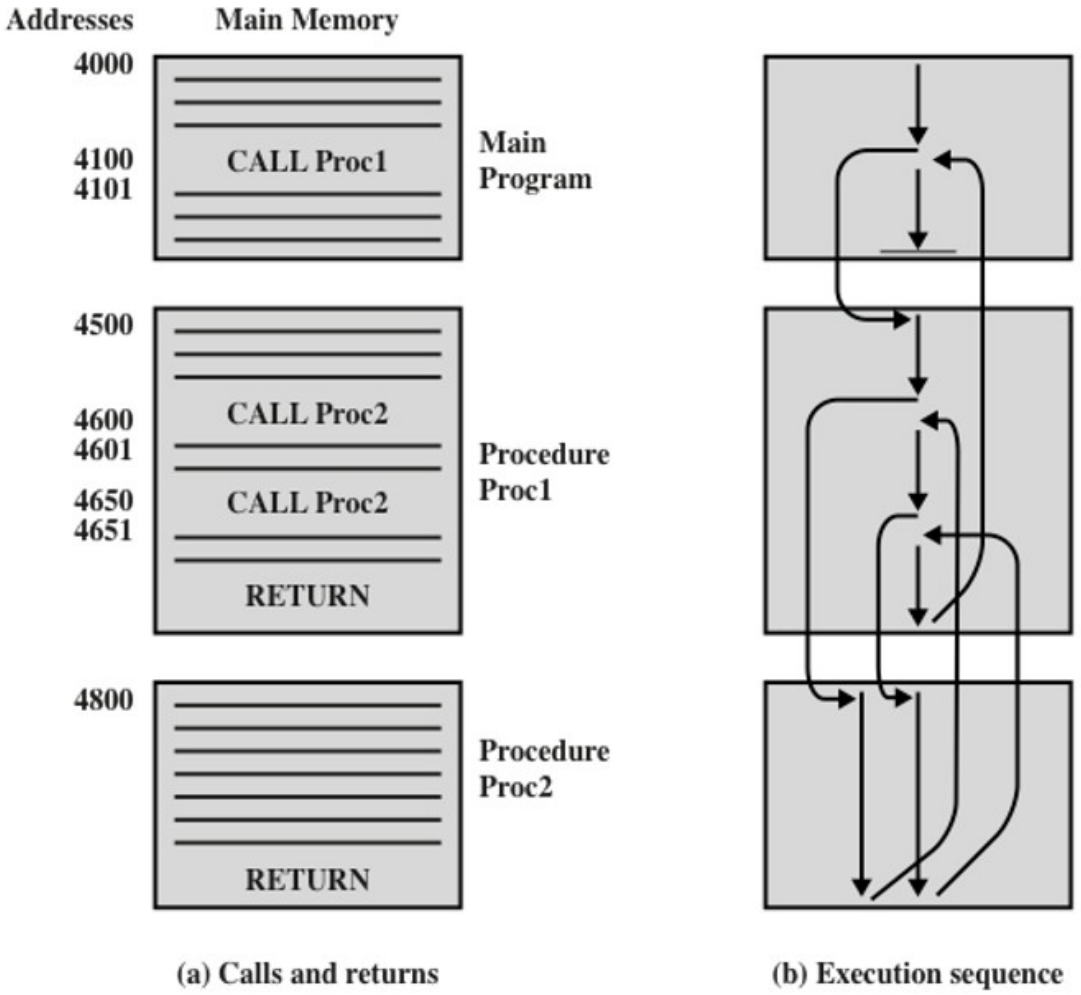
So there are two operations on stacks:

- *Push* --- add a data element.
- *Pop* --- remove the last added element (if any).



Sequence:
Push 1
Push 2
Push 3
Pop
Push 4
Pop
Pop
Pop

The use of procedures to construct a program



In this example, there is a main program starting at location 4000. This program includes a call to procedure Proc1, starting at location 4500.

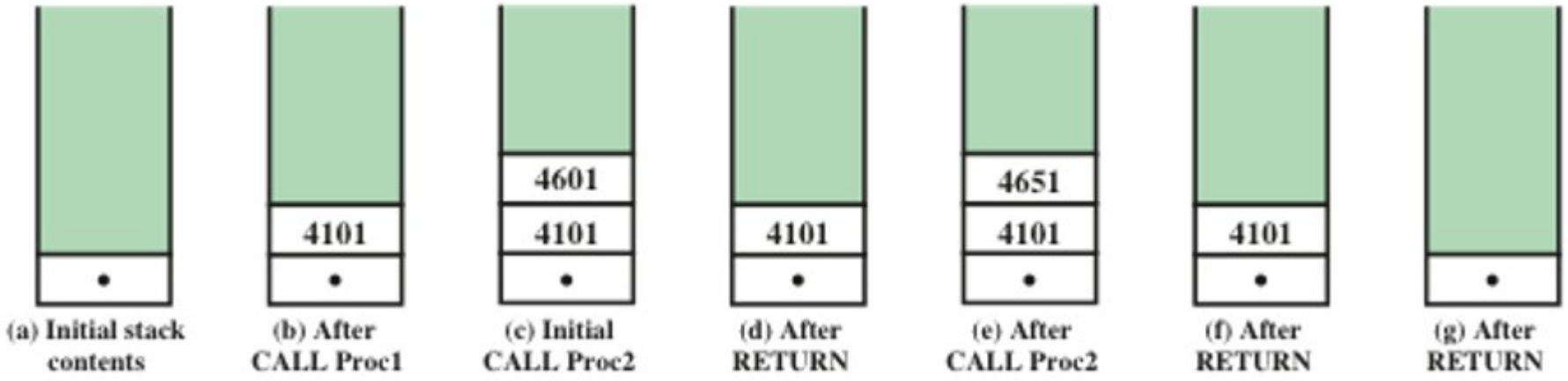
When this call instruction is encountered, the processor suspends execution of the main program and begins execution of Proc1 by fetching the next instruction from location 4500.

Within Proc1, there are two calls to Proc2 at location 4800. In each case, the execution of Proc1 is suspended and Proc2 is executed.

The RETURN statement causes the processor to go back to the calling program and continue execution at the instruction after the corresponding CALL instruction. This behavior is illustrated with arrows on (b).

Source: Stallings book (fig 13.8)

Use of stack to implement nested procedures



When the processor executes a call, it places the return address on the stack.
When it executes a return, it uses the address on the stack.

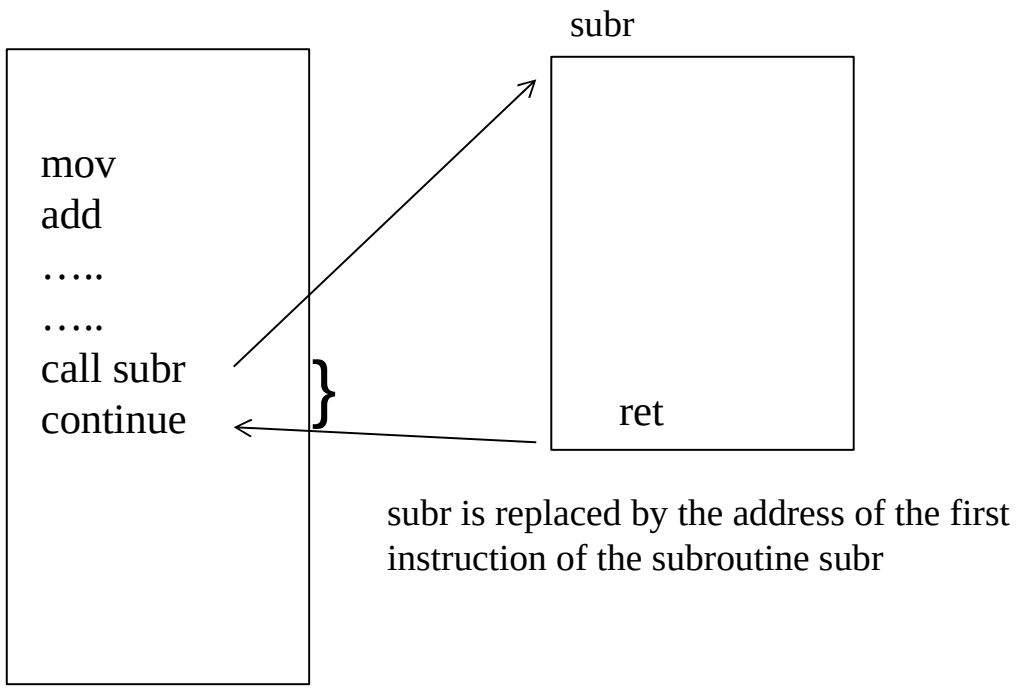
Source: Stallings book (fig 13.9)

Control instructions: Subroutines

C

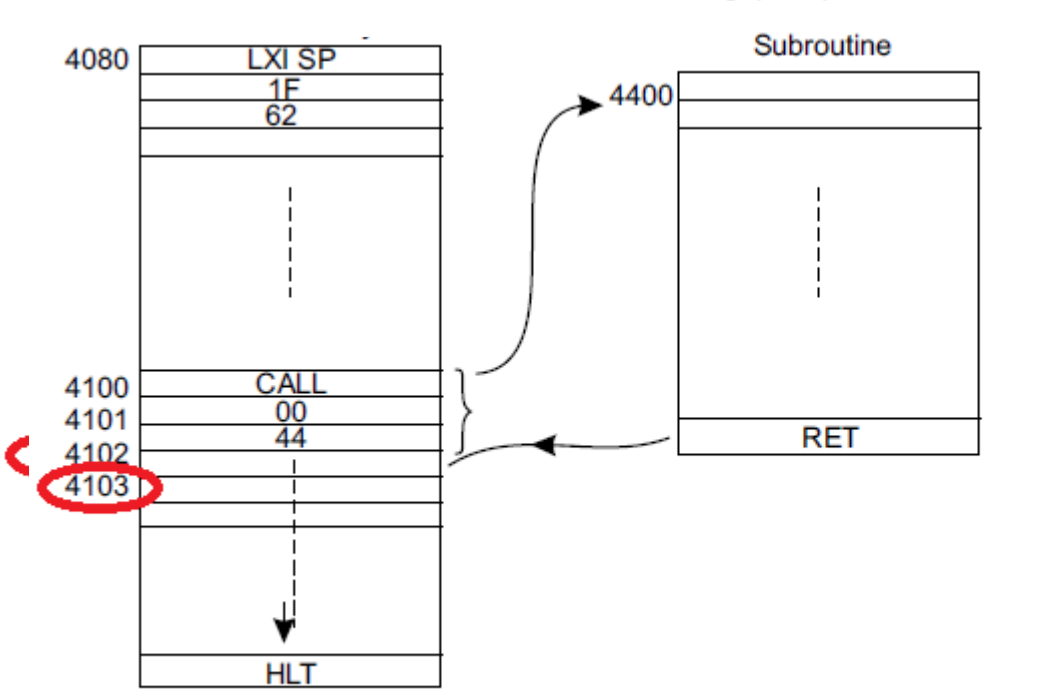
```
int var1, var2;

main()
{
   var1=0;
   var2=var1++;
   subr();
   …

}


subr()
{
    …
}
```

Assembly



subr is replaced by the address of the first instruction of the subroutine subr

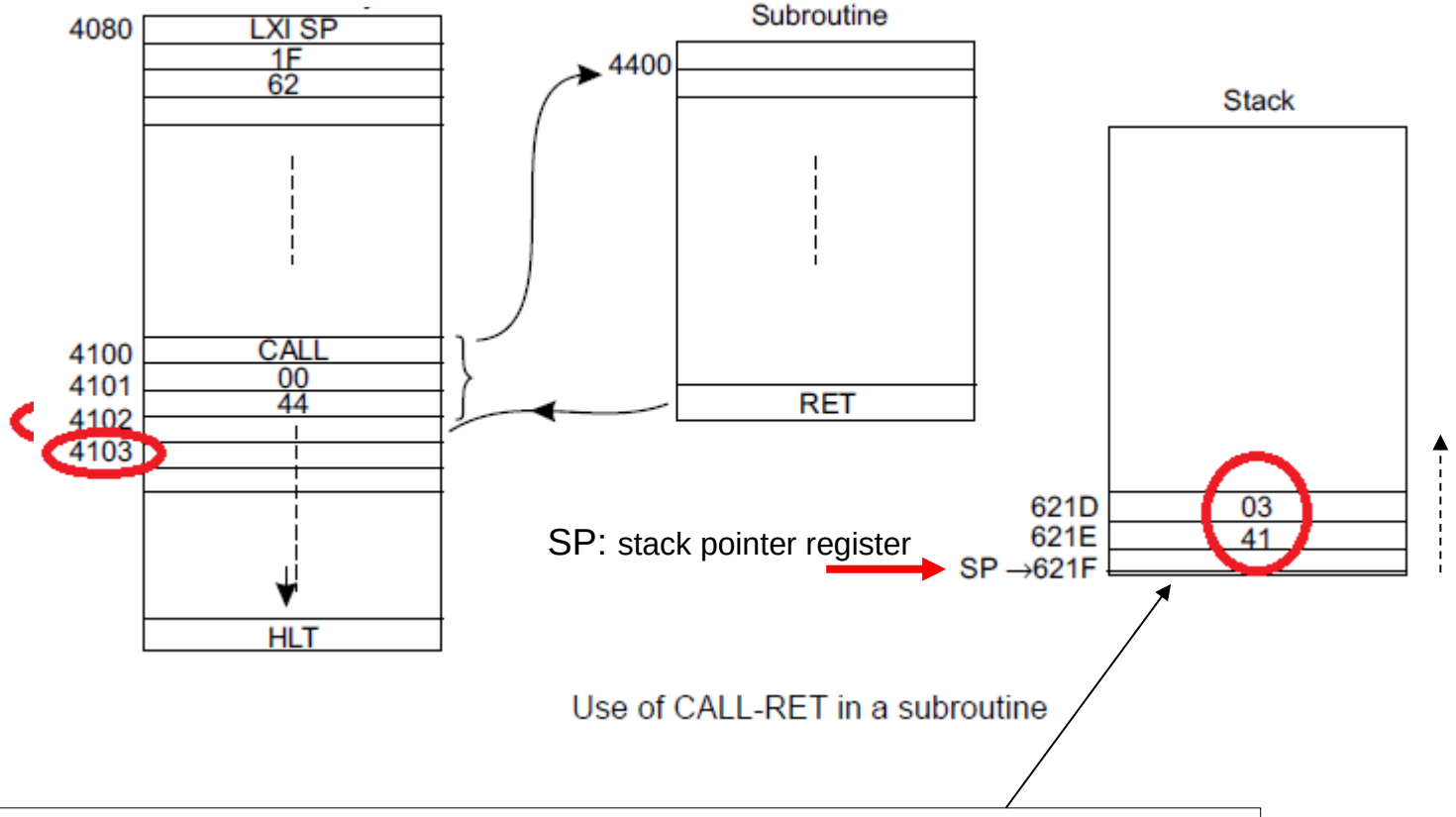
Control instructions: Subroutines



Use of CALL-RET in a subroutine

2. Control instructions: Subroutines

- In order to return to the right point, the contents of the PC must be stored when a subroutine is called.
- The solution is to store return address on a call **stack**.



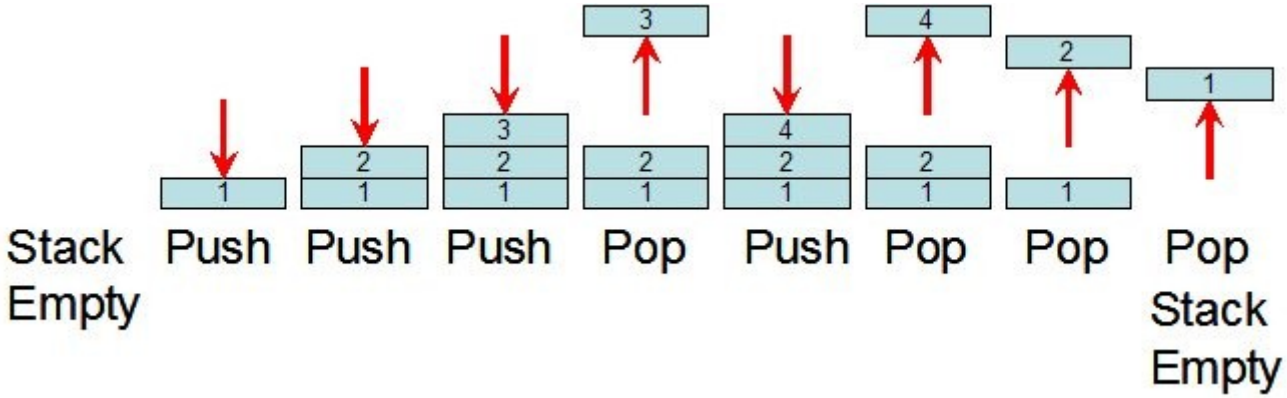
Normally, stack starts at high address in memory and grows from higher addresses to lower ones

Stack

“Last in First out” (LIFO) principle.

So there are two operations on stacks:

- *Push* --- add a data element.
- *Pop* --- remove the last added element (if any).



Sequence:
Push 1
Push 2
Push 3
Pop
Push 4
Pop
Pop
Pop

Implementing stacks

Here we assume a stack that grows from higher addresses to lower ones

A stack may be implemented with:

- A block of successive memory locations storing the contents of the stack (it's usual that stack starts at high addresses)
- A register, called the *stack pointer (SP)* storing the address of the "top" of the stack.

- To *push* a value onto the stack, decrement the address in the stack pointer and store it there.

- To *pop* a value from the stack, dereference (get the content of) the address stored in the stack pointer, which is then incremented by one.

Implementing Subroutines

We can now implement subroutines using stacks:

- To *call a* subroutine, the contents of the program counter are pushed onto the stack and the PC is reset to the starting address of the subroutine.

- To *return,* the program counter is reset to the value popped from the stack.

In this way, we can nest subroutine calls *to any depth* (provided we have enough storage --- otherwise we will have *stack overflow).*

In fact, we typically need to store more information ---e.g. contents of data registers --- for each subroutine call: the *stack frame for the call.*

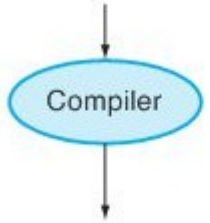
The stack frame generally includes the following components:

- The return address
- Argument variables passed on the stack
- Local variables (in high level languages)
- Saved copies of any registers modified by the subprogram that need to be restored

Extra: Relationship between Compiler and Assembler

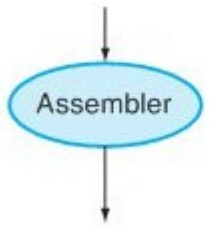
High level language (e.g. C)

```
swap(int v[], int k)
{int temp;
   temp = v[k];
   v[k] = v[k+1];
   v[k+1] = temp;
}
```



Assembly language (e.g. MIPS)

```
swap:
        multi $2, $5,4
        add   $2, $4,$2
        lw    $15, 0($2)
        lw    $16, 4($2)
        sw    $16, 0($2)
        sw    $15, 4($2)
        jr    $31
```



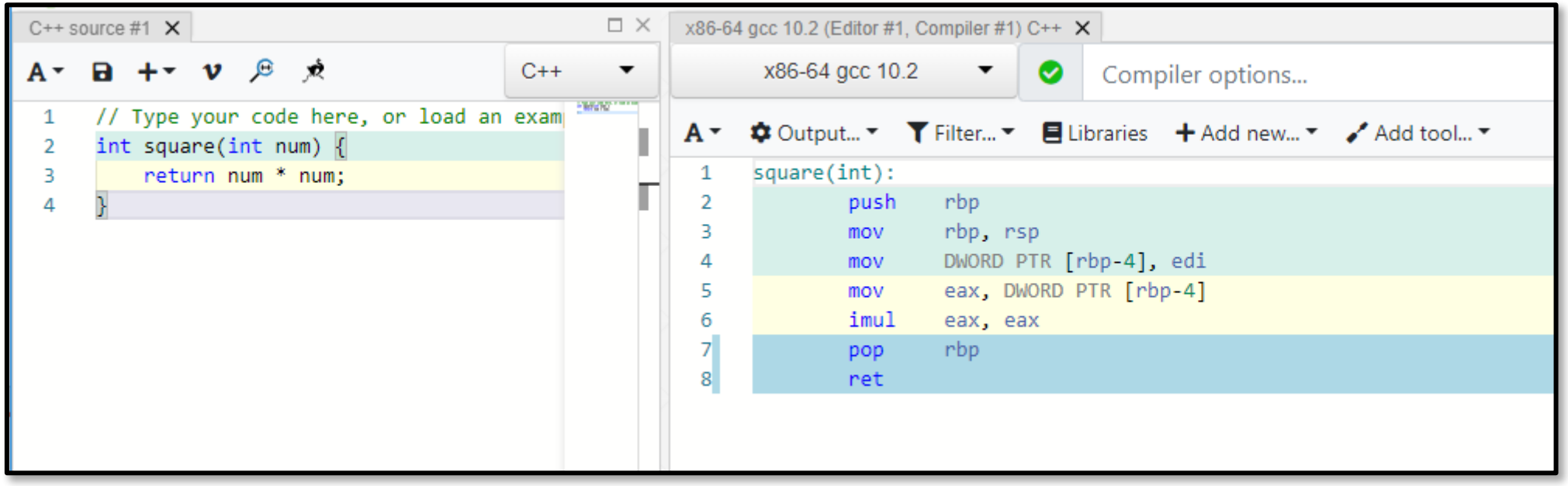
Binary Machine code

```
00000000101000100000000100011000
00000000100000100001000000100001
10001101111000100000000000000000
10001111000010010000000000000100
10101111000010010000000000000000
10101101111000100000000000000100
00000011111000000000000000001000
```

Extra: x86-64 and **Compiler Explorer**

Stack frame on x86-64: https://eli.thegreenplace.net/2011/02/04/where-the-top-of-the-stack-is-on-x86/

https://gcc.godbolt.org/



Extra: if you want to learn more about how code is generated from a C program, visit https://www.recurse.com/blog/7-understanding-c-by-learning-assembly

Summary

In this lecture we have introduced:

1. Design philosophies for instruction sets: RISC vs. CISC

2. Control instructions:
   a) unconditional branching
   b) conditional branching and
   c) subroutines

Next

- Faster execution via Instruction level parallelism: pipelining
  - Next: superscalar architectures.
  - I/O (input/output)